

# Scaler des Microservices avec les Composants Spring Cloud

# Plan

**A. Introduction**

**B. Qu'est-ce que Spring Cloud ?**

**C. À quoi ça sert ?**

**D. Les Défis du Scaling et les Solutions Spring Cloud**

**E. Spring Cloud Eureka - La Découverte de Services**

**F. Spring Cloud Config - La Configuration Centralisée**

**G. Spring Cloud Gateway & LoadBalancer - Le Routage**

**H. Spring Cloud Circuit Breaker - La Résilience**

**Conclusion**

# A. Introduction

## Contexte:

Pour gérer des microservices à l'échelle d'Internet, il faut disposer de plus de fonctionnalités que celles offertes par le framework **Spring Boot**.

- Le projet **Spring Cloud** propose un ensemble de composants spécialement conçus pour atteindre ces fonctionnalités supplémentaires sans effort.

Ce chapitre fournira un aperçu approfondi des différents composants du projet **Spring Cloud**, tels que **Eureka**, **Zuul**, **Ribbon** et **Spring Config**, en les positionnant par rapport au modèle de capacité des microservices.

# A. Introduction – (suite)

## Objectif:

Ce chapitre nous permettra de voir les éléments suivants :

- Le ***Spring Config*** Server pour externaliser la configuration
- Le ***serveur Eureka*** pour l'enregistrement et la découverte des services Comprendre l'importance de ***Zuul*** en tant que proxy de service et passerelle
- La mise en œuvre de ***l'enregistrement automatique*** des microservices et de la découverte des services
- Le ***messaging Spring*** Cloud pour la composition réactive asynchrone des microservices

## B. A Propos de Spring Cloud

## B. Qu'est-ce que Spring Cloud ?

- **Spring Cloud:** est un ensemble d'outils et de frameworks qui s'appuie sur [Spring Boot](#) pour simplifier le développement d'applications distribuées et de systèmes de microservices.

Il offre des solutions pour gérer des problèmes **courants des architectures réparties**, comme la configuration centralisée, la découverte de services et la tolérance aux pannes.

Cet outil est destiné aux développeurs construisant des **applications Java distribuées**.

## B. Qu'est-ce que Spring Cloud ? (suite)

### ❑ Ecosystème

Il s'appuie sur des projets éprouvés de **Netflix** (Eureka, Hystrix, Zuul) et d'autres (**Consul**, **Zookeeper**) en fournissant une abstraction Spring pour une intégration transparente.

# ■ À quoi ça sert ?

**Spring Cloud** permet de simplifier le développement d'architectures complexes où de nombreux microservices doivent communiquer entre eux.

## Composants principaux :

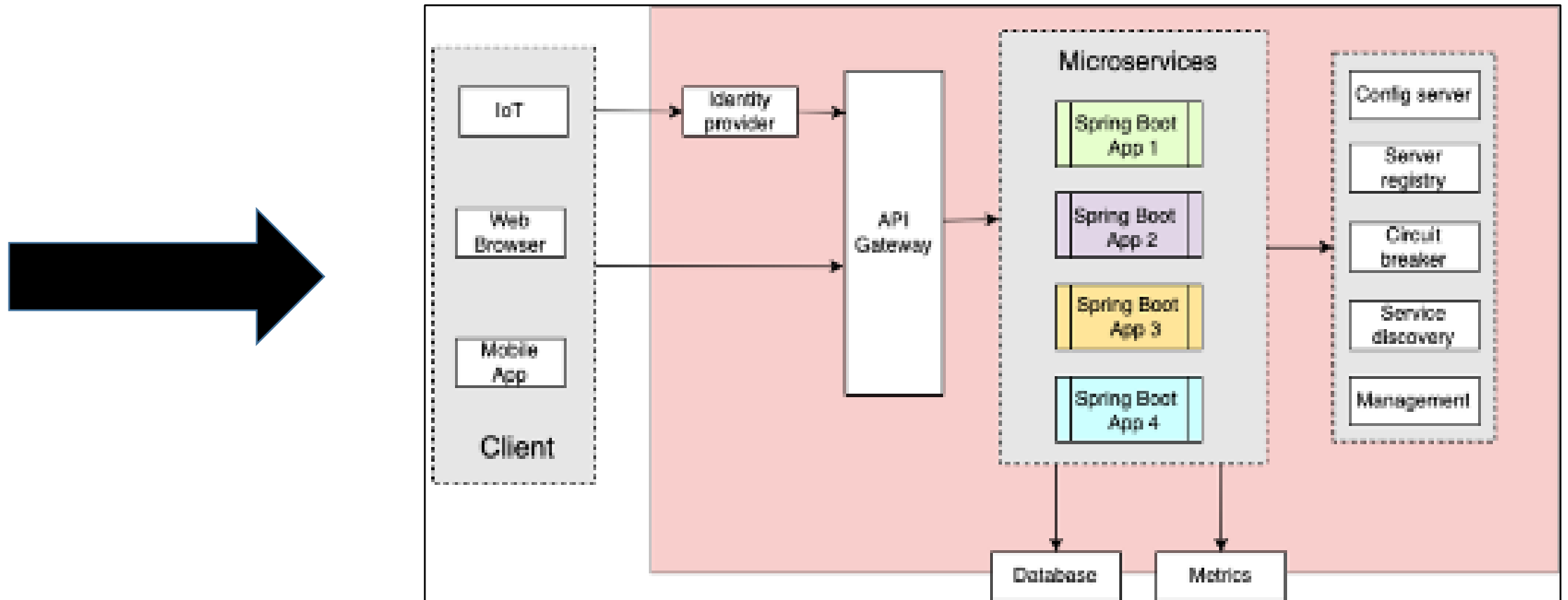
- **Spring Cloud Config** : Permet de centraliser la configuration de tous les microservices.
- **Service Discovery** : Aide les services à se trouver mutuellement dans un système distribué (par exemple, avec Eureka).
- **Intelligent Routing** : Permet d'acheminer les requêtes vers les services appropriés (par exemple, avec [Spring Cloud Gateway](#) et [Feign](#)).
- **Circuit Breakers** : Aide à gérer les défaillances en évitant qu'elles n'affectent l'ensemble du système (par exemple, avec Hystrix).

## Avantages :

- Réduit la complexité des architectures de microservices.
- Fournit des modèles et des outils pour des tâches courantes, ce qui accélère le développement.
- Permet la gestion dynamique de la configuration sans redémarrage des services.
- Facilite l'intégration avec les fournisseurs de cloud comme AWS et Google Cloud



## ■ Schéma illustratif

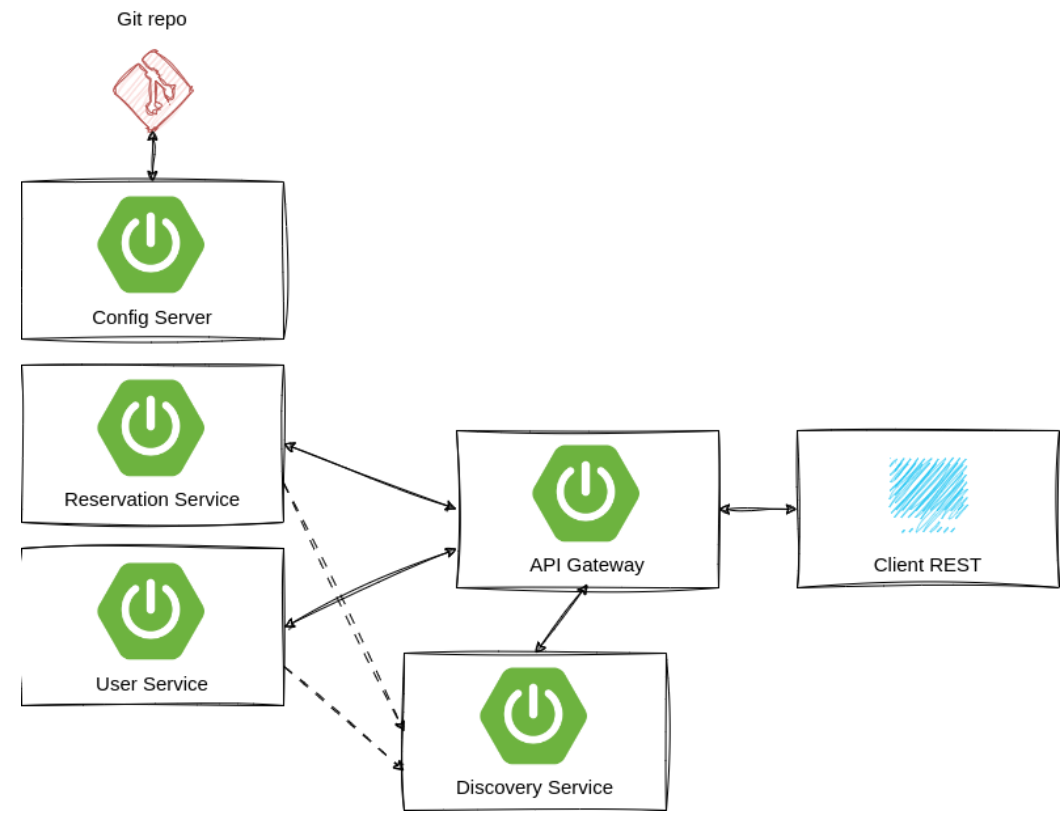


## C. Les Défis du Scaling et les Solutions Spring Cloud

# C. Les Défis du Scaling et les Solutions Spring Cloud

## Les défis du scaling incluent:

- la complexité de l'architecture distribuée,
- la gestion de la cohérence des données,
- la dépendance à l'infrastructure réseau,
- et la sécurité.



# C. Les Défis du Scaling et les Solutions Spring Cloud

(suite)

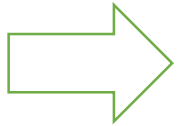


## ❑ Défis du scaling

- **Complexité architecturale** : L'ajout de nombreux serveurs dans une architecture distribuée demande des compétences avancées.
- **Cohérence des données** : Maintenir la cohérence des données entre différentes bases de données ou instances est difficile dans un système distribué.
- **Performances réseau** : Les performances dépendent d'une infrastructure réseau fiable ; des goulots d'étranglement peuvent survenir.
- **Gestion des pannes** : Le risque de défaillance en cascade peut survenir si un composant tombe en panne, affectant l'ensemble du système.
- **Sécurité** : Avec plus de ressources et de services, la surface d'attaque potentielle augmente.

# C. Les Défis du Scaling et les Solutions Spring Cloud (suite)

❑ Solutions proposées  
par Spring Clouds



- **Découverte de services** : Spring Cloud permet aux services de s'enregistrer et de se trouver les uns les autres dynamiquement, éliminant le besoin de configurer manuellement les adresses des services.
- **Gestion de la configuration** : Spring Cloud permet de centraliser la configuration des microservices, la rendant gérable et modifiable indépendamment de chaque service.
- **Équilibrage de charge et passerelles** : Des outils comme [Spring Cloud Gateway](#) peuvent équilibrer la charge en répartissant les requêtes entre plusieurs instances d'un service, améliorant la performance et la disponibilité.
- **Tolérance aux pannes** : Spring Cloud intègre des solutions comme les "circuit breakers" (par ex. [Resilience4j](#)) pour empêcher les défaillances de se propager, et des mécanismes de "retry" pour réessayer les appels en cas d'échec temporaire.
- **Surveillance** : L'intégration avec des outils de surveillance comme [Spring Cloud Sleuth](#) (qui s'intègre avec [Zipkin](#) ou [Jaeger](#)) facilite le suivi des requêtes à travers les microservices.

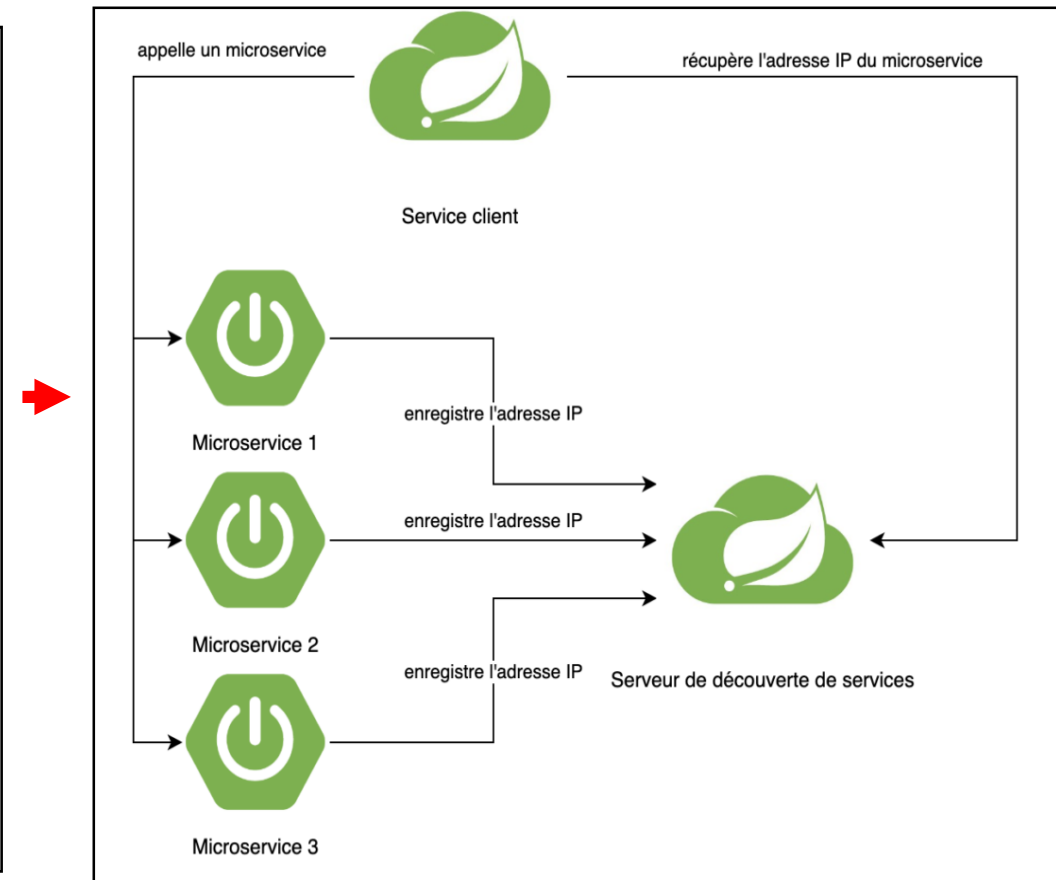
## E. Spring Cloud Eureka - La Découverte de Services

# E. Spring Cloud Eureka - La Découverte de Services

**Spring Cloud Eureka** est un composant clé de la découverte de services dans les architectures de **microservices**.

Il permet aux applications de s'enregistrer auprès d'un **serveur Eureka** (le registre de services) et de **découvrir dynamiquement d'autres services** sans avoir à coder leurs adresses en dur.

Cela est rendu possible grâce à une approche côté client où chaque **microservice (client Eureka)** *s'enregistre et recherche d'autres services via le serveur Eureka*.



# E. Spring Cloud Eureka - La Découverte de Services (suite)

## ❑ Son Fonctionnement

Le fonctionnement de Spring Cloud Eureka repose sur trois étapes principales :

### 1. Enregistrement (Registration)

Au démarrage, chaque microservice configuré comme **Eureka Client** contacte le **Eureka Server** et lui fournit **ses métadonnées** (nom du service, IP, port, état de santé).

Le service s'enregistre lui-même dans le registre, déclarant ainsi sa disponibilité.

### 2. Renouvellement (**Renewals ou Heartbeats**)

Pour s'assurer que le registre est à jour, chaque client envoie régulièrement (par défaut toutes les **30 secondes**) un **"battement de cœur" (heartbeat)** au serveur.

Cela indique au serveur que le service est toujours en ligne et opérationnel. Si un serveur ne reçoit pas de **heartbeat** pendant une période prolongée, il retire l'instance du registre (le délai d'expiration par défaut est de 90 secondes).



# E. Spring Cloud Eureka - La Découverte de Services (suite)

## ❑ Son Fonctionnement (Suite)

Le troisièmeme étapes :

### 3. Récupération (Fetching Registry) et Invocation

Lorsqu'un microservice a besoin de communiquer avec un autre, il ne se connecte pas directement. Il interroge plutôt le **Eureka Server** pour obtenir la liste des instances disponibles pour le service cible.

Le client met en cache cette liste localement. Grâce à des **bibliothèques côté client** comme **Ribbon** (souvent utilisées conjointement avec **Eureka**, ou remplacées par des alternatives plus modernes intégrées dans **Spring Cloud LoadBalancer**), la charge est équilibrée (**load balancing**) entre les différentes instances disponibles, et les communications peuvent avoir lieu.

# E. Spring Cloud Eureka - La Découverte de Services (suite)

## ❑ Caractéristiques clés

### **Architecture "Peer-to-Peer" :**

Il est courant de déployer plusieurs serveurs Eureka qui se synchronisent entre eux. Si un serveur tombe en panne, le système continue de fonctionner.

### **Résilience Client (Client-Side Resilience) :**

La logique de découverte et le cache se trouvent côté client. Si le serveur Eureka tombe en panne temporairement, les clients peuvent toujours utiliser leur cache local pour trouver des services, rendant le système très tolérant aux pannes du registre lui-même.

### **Simple à mettre en œuvre :**

L'ajout des annotations `@EnableEurekaServer` et `@EnableDiscoveryClient` ou `@EnableEurekaClient` (selon la version de Spring Cloud) suffit pour activer la fonctionnalité.

# ■ Configuration de base

## Côté Serveur (pom.xml)

xml

```
<dependencies> <dependency>  
<groupId>org.springframework.cloud</groupId>  
<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>  
</dependency> </dependencies>
```

## Côté Serveur (Application.properties) Properties

```
server.port=8761 eureka.instance.hostname=localhost  
eureka.client.register-with-eureka=false  
eureka.client.fetch-registry=false
```

## Côté Client (pom.xml)

xml

```
<dependencies> <dependency>  
<groupId>org.springframework.cloud</groupId>  
<artifactId>spring-cloud-starter-netflix-eureka-  
client</artifactId>  
</dependency> <!-- Pour les versions modernes de  
Spring Cloud, on utilise souvent 'spring-cloud-starter-  
loadbalancer' explicitement ou implicitement -->  
</dependencies>
```

## Côté Client (Application.properties) Properties

```
spring.application.name=mon-service-produit  
server.port=8080  
eureka.client.serviceUrl.defaultZone=http://localhost:87  
61/eureka/
```

# ■ Recap sur Spring Cloud Eureka

Spring Cloud Eureka est donc un pilier essentiel pour construire des systèmes de microservices flexibles, résilients et évolutifs en gérant de manière transparente la complexité liée à la localisation dynamique des services sur le réseau.

## **Avantage pour le Scaling**

- Dynamique : Les nouvelles instances sont automatiquement découvertes
- Découplage : Les services n'ont pas besoin de connaître les adresses IP et ports des autres services

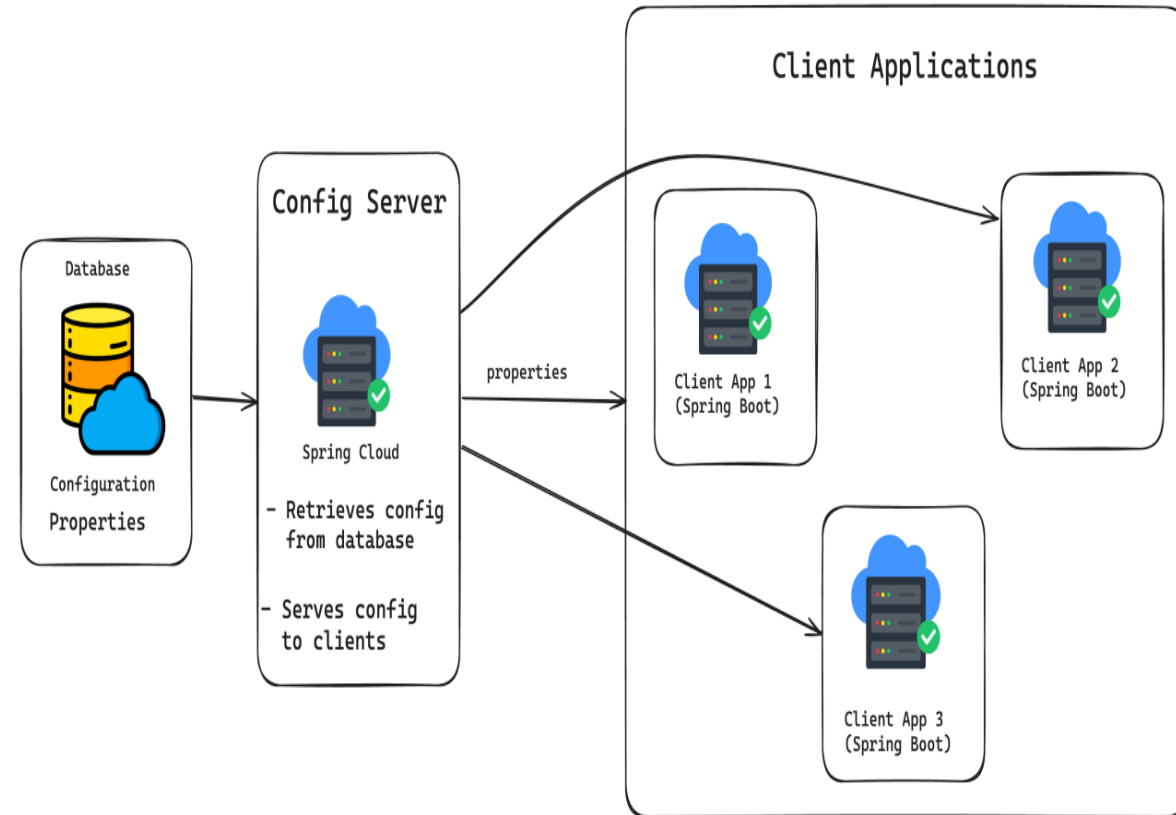
## F. Spring Cloud Config - La Configuration Centralisée

## F. Spring Cloud Config - La Configuration Centralisée

**Spring Cloud Config** est une solution de configuration centralisée pour les systèmes distribués, comme les **microservices**.

Elle consiste en un serveur de configuration qui stocke les propriétés dans un dépôt centralisé (**souvent Git**) et en des applications clientes qui interrogent le serveur pour récupérer leurs configurations spécifiques.

Cela simplifie la gestion des configurations, permet leur rafraîchissement dynamique sans redémarrage et assure la cohérence entre les différentes applications.



## F. Spring Cloud Config - La Configuration Centralisée (suite)

### ❑ Son Fonctionnement

**Serveur de configuration** : Un service dédié, généralement une application Spring Boot avec l'annotation **@EnableConfigServer**, qui sert les configurations via une **API REST**.

**Dépôt de configuration** : Un endroit unique pour stocker tous les fichiers de configuration. Git est un choix courant pour le versionnement, mais d'autres options comme SVN ou le système de fichiers sont aussi possibles.

**Clients** : Les applications clientes (**microservices**) se connectent au serveur au démarrage pour récupérer leur configuration spécifique, basée sur leur nom et leur profil (par exemple, **application-prod.properties**).

**Rafraîchissement dynamique** : Les applications clientes peuvent être configurées pour rafraîchir leur configuration à chaud, sans nécessiter un redémarrage complet, grâce à des fonctionnalités comme **@RefreshScope**

## ■ Recap sur Spring Cloud Config - La Configuration Centralisée

### Avantage pour le Scaling

**Gestion centralisée** : Évite la duplication des configurations et facilite leur gestion dans une architecture de microservices.

**Cohérence** : Assure que toutes les applications utilisent les mêmes versions de configuration et les mêmes paramètres.

**Versionnement** : L'utilisation d'un dépôt de versionnement permet de suivre les modifications et de revenir à une version précédente si nécessaire.

**Flexibilité** : Permet de gérer des configurations spécifiques à chaque environnement (développement, production, etc.).



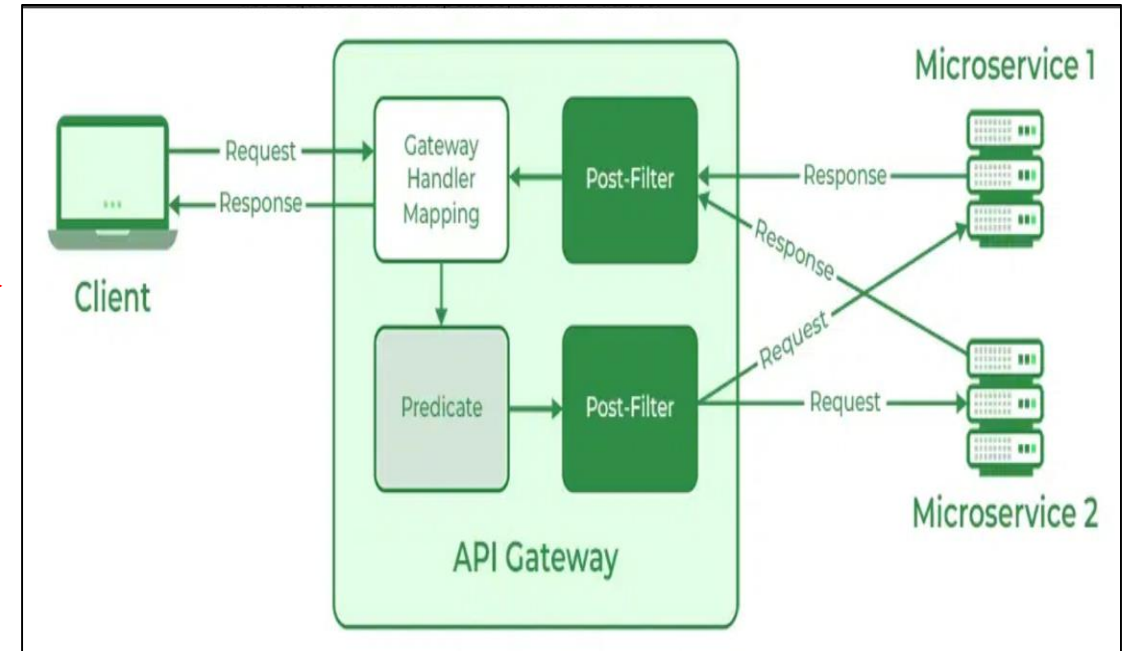
## G. Spring Cloud Gateway & LoadBalancer - Le Routage

# G. Spring Cloud Gateway & LoadBalancer - Le Routage

[Spring Cloud Gateway](#) et [Spring Cloud LoadBalancer](#) travaillent ensemble pour le routage et l'équilibrage de charge dans une architecture de microservices.

**Spring Cloud Gateway** agit comme un point d'entrée unique pour les requêtes, gérant les règles de routage dynamique, la sécurité et d'autres préoccupations transversales. ➔

**Spring Cloud LoadBalancer** gère la distribution du trafic entre les instances des microservices



# G. Spring Cloud Gateway & LoadBalancer - Le Routage (suite)

## ❑ Son Fonctionnement

**Spring Cloud Gateway** s'intègre parfaitement Spring Cloud LoadBalancer pour l'équilibrage de charge côté client .

### 1- Dépendance :

Pour activer cette fonctionnalité, vous devez généralement vous assurer ***org.springframework.cloud:spring-cloud-starter-loadbalancer*** que le fichier se trouve dans le ***classpath***.

### 2- Configuration :

Dans la configuration de votre passerelle (par exemple, `application.yml` ou la configuration Java), vous utilisez le **lbschéma URI** spécial suivi du nom de service enregistré au lieu d'un hôte et d'un port concrets.

```
spring: cloud:
  gateway: routes:
    - id: api_route
    - uri: lb://my-service-name # The 'lb' scheme enables load balancing
    - predicates: - Path=/api/**
```

# G. Spring Cloud Gateway & LoadBalancer - Le Routage (suite)

## ❑ Son Fonctionnement (suite)

### 3- Processus :

Lorsqu'une requête arrive à la passerelle correspondant à cet itinéraire :

La passerelle utilise l' *DiscoveryClientIntégration* pour récupérer les instances disponibles de « *my-service-name* » à partir du registre de services.

**Spring Cloud LoadBalancer** (Par défaut, en utilisant un algorithme Round-Robin) sélectionne une instance saine.

La passerelle achemine la requête vers l'instance choisie

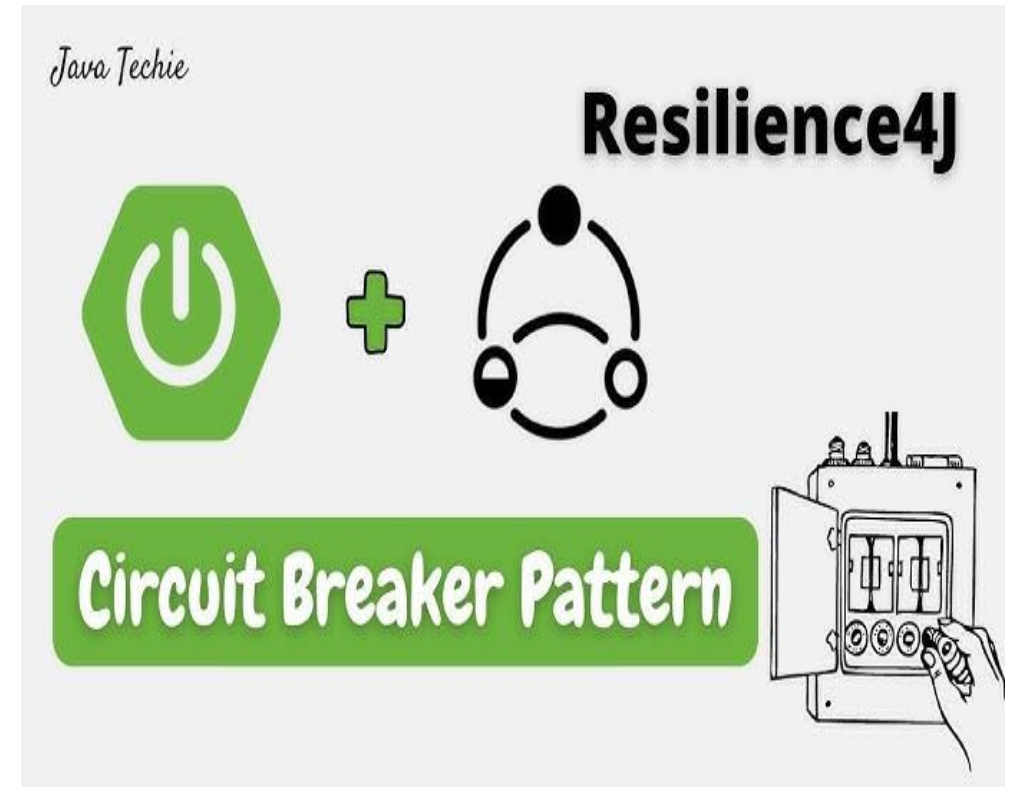
- ❑ L' intégration simplifie le routage et assure une distribution efficace du trafic à travers des **microservices** évolutifs, ce qui est un élément clé des applications cloud-native robustes

## H. Spring Cloud Circuit Breaker - La Résilience

# H. Spring Cloud Circuit Breaker - La Résilience

**Spring Cloud Circuit Breaker** est une bibliothèque qui fournit une couche d'abstraction pour intégrer le [motif de disjoncteur](#) dans les applications, renforçant ainsi leur **résilience** face aux défaillances de services.

Il permet de prévenir les **défaillances en chaîne** (ou en cascade) en interceptant les appels vers un service défaillant. Le motif fonctionne avec trois états : "fermé" (où les appels réussissent), "ouvert" (où les appels sont immédiatement rejetés pour éviter une surcharge), et "semi-ouvert" (où des requêtes de test sont autorisées pour vérifier le rétablissement du service).



## H. Spring Cloud Circuit Breaker - La Résilience (suite)

### ❑ Son Fonctionnement

- **État fermé :**  
Le disjoncteur autorise les appels et surveille les échecs.
- **État ouvert :**  
Si le nombre d'échecs dépasse un seuil, le circuit s'ouvre. Il bloque immédiatement toutes les requêtes vers le service défaillant et exécute une méthode de repli (fallback) si elle est définie.
- **État semi-ouvert :**  
Après une période d'attente configurable, le disjoncteur autorise un nombre limité de requêtes de test.
- **Réinitialisation :**  
Si les requêtes de test réussissent, le circuit se referme. Sinon, il se rouvre immédiatement pour une autre période d'attente.

# CONCLUSION



# □ Conclusion

## **Récapitulatif**

Spring Cloud fournit une boîte à outils complète pour les défis opérationnels des microservices.

## **Synergie**

Les composants fonctionnent ensemble pour créer un système dynamique, configurable et résilient.

## **Bénéfice Final**

Mise à l'échelle horizontale (scale-out) en toute confiance avec adaptation automatique.

## **Perspective**

Évolution vers des solutions modernes comme Spring Cloud Gateway, LoadBalancer et Resilience4j.



MERCI !